

Evolução da Maturidade de Desenvolvimento em Projetos Spring Boot

Gabriel Ferreira¹, Guilherme Augusto¹

¹Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Belo Horizonte – MG – Brazil

{gabriel.amaral.1447804, gajsouza}@sga.pucminas.br

Abstract. *Internal software quality in object-oriented systems can be observed through structural metrics of coupling, complexity, inheritance, cohesion and size. In the Java ecosystem, Spring Boot is a widely adopted framework, yet longitudinal evidence on how the structural quality of public GitHub projects evolves—and whether the generative-AI era (2022–present) coincides with measurable change—remains scarce. This work aims to analyse that evolution across four temporal cohorts defined by repository creation date. Object-oriented metrics will be extracted with the CK tool, organised through an ETL pipeline in Power BI and compared with non-parametric tests (Kruskal–Wallis, Mann–Whitney, Bonferroni correction) in reproducible Python/R notebooks.*

Resumo. *A qualidade interna de software orientado a objetos pode ser observada por métricas estruturais de acoplamento, complexidade, herança, coesão e tamanho. No ecossistema Java, o Spring Boot é um framework de ampla adoção; contudo, ainda são escassas evidências longitudinais sobre como a qualidade estrutural de projetos públicos no GitHub evolui e se a Era da IA generativa (2022–atual) coincide com alterações mensuráveis nessas métricas. O objetivo deste trabalho é analisar essa evolução em quatro coortes temporais definidas pela data de criação dos repositórios. As métricas serão extraídas com a ferramenta CK, organizadas em um pipeline de ETL no Power BI e comparadas por testes não paramétricos (Kruskal–Wallis, Mann–Whitney e correção de Bonferroni) em notebooks reproduzíveis em Python/R.*

Bacharelado em Engenharia de Software - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia - danilomaia@pucminas.br
Orientador de conteúdo (TCC I): João Pedro Batisteli - joabatisteli@pucminas.br
Orientador de conteúdo (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador acadêmico (TCC I): Cleiton Tavares - cleitontavares@pucminas.br
Orientador do TCC II: (A ser definido no próximo semestre)

Belo Horizonte, 24 de agosto de 2026.

1. Introdução

A qualidade interna de *software* descreve propriedades estruturais do código-fonte que podem ser observadas sem a execução do programa, tais como modularidade, coesão, acoplamento e facilidade de manutenção [ISO/IEC 2011, Pressman and Maxim 2020]. Em

sistemas orientados a objetos, essas propriedades têm sido operacionalizadas pela suíte de métricas proposta por Chidamber e Kemerer, que quantifica acoplamento, complexidade, herança e coesão em nível de classe [Chidamber and Kemerer 1994]. No ecossistema Java, o *framework* Spring Boot consolidou convenções de injeção de dependências, organização em camadas e autoconfiguração, tornando-se um recorte representativo para estudos empíricos sobre estrutura de código em repositórios públicos [Sun et al. 2022]. Importa notar, contudo, que o Spring Boot atua como um mecanismo de **acoplamento vertical** controlado: a injeção de dependências (IoC) resolve automaticamente as ligações entre camadas (controlador, serviço, repositório), mas não governa a qualidade interna dos métodos, a coesão das classes nem o acoplamento horizontal entre módulos de domínio. Por isso, as métricas que mais claramente refletem as decisões do desenvolvedor — ou da IA que gera o código — são **WMC** (peso computacional dos métodos), **LCOM_HS** (coesão interna das classes) e o **CBO horizontal** (acoplamento entre pacotes de negócio não intermediados pelo *framework*). A combinação entre essas métricas e a ampla disponibilidade de projetos Spring Boot no GitHub permite examinar, com evidências extraídas do próprio código, como desenvolvedores e agentes de IA organizam as suas aplicações.

Apesar da consolidação dessas métricas e da popularidade do Spring Boot, a maior parte das investigações empíricas sobre qualidade estrutural em Java adota recortes transversais, isto é, analisa um conjunto de sistemas em um único instante, sem contrastar gerações distintas de projetos ao longo do tempo [Child et al. 2019, El Emam et al. 2001]. Paralelamente, a partir de 2022, o desenvolvimento passou a incorporar de forma intensiva assistentes de inteligência artificial generativa, como GitHub Copilot, ChatGPT, Claude Code e Cursor, o que alterou o ambiente de produção de código [Peng et al. 2023, Yetistiren et al. 2023]. **O problema investigado neste trabalho é a ausência de evidências empíricas longitudinais sobre como a qualidade estrutural de projetos Spring Boot hospedados no GitHub evolui ao longo de diferentes épocas históricas e, em particular, se a Era da IA assistida introduziu alterações significativas nas métricas de peso dos métodos (WMC), de coesão interna das classes (LCOM) e de acoplamento horizontal entre módulos de domínio (CBO) — dimensões sobre as quais o *framework* não exerce controle direto e que, portanto, expressam a qualidade do código escrito pelo desenvolvedor ou gerado pela IA.**

A relevância de tratar esse problema decorre de três fatores. Primeiro, a qualidade interna está associada ao esforço de manutenção e à viabilidade de longo prazo dos sistemas [Pressman and Maxim 2020, ISO/IEC 2011]; portanto, desconhecer a trajetória estrutural de um ecossistema tão difundido quanto o Spring Boot limita decisões de prática e de pesquisa. Segundo, a mineração de repositórios de *software* no GitHub oferece base empírica em larga escala, desde que critérios de seleção, exclusão de *forks* e maturidade temporal sejam observados [Hassan 2008, Kalliamvakou et al. 2016]. Terceiro, evidências recentes mostram que assistentes de IA reduzem o tempo de conclusão de tarefas e alteram a qualidade do código gerado em experimentos controlados [Peng et al. 2023, Moradi Dakhel et al. 2023, Liu et al. 2023], mas ainda não esclarecem se esses efeitos se manifestam nas métricas estruturais de projetos reais publicados na plataforma. Crucialmente, como o Spring Boot não governa a construção interna dos métodos, a coesão das classes nem o acoplamento horizontal entre módulos de domínio, qualquer variação observada em WMC, LCOM_HS e CBO no eixo E3 → E4 será atribuível ao estilo de produção de código — humano ou assistido por IA — e não

ao próprio *framework*. Sem uma análise por coortes, permanece indefinido se a Era da IA coincide com estabilidade, melhoria ou degradação dessas métricas.

O objetivo geral deste trabalho é analisar longitudinalmente a evolução da qualidade estrutural de projetos Spring Boot *open source* hospedados no GitHub, organizados em quatro coortes temporais, com ênfase nas métricas de peso dos métodos (WMC), coesão interna das classes (LCOM_HS) e acoplamento horizontal entre módulos de domínio (CBO), investigando se a Era da IA assistida introduziu alterações observáveis nessas dimensões em relação ao código produzido exclusivamente por desenvolvedores humanos. Os objetivos específicos são: (i) caracterizar a evolução do acoplamento entre classes ao longo das quatro épocas; (ii) caracterizar a evolução da complexidade e do tamanho de código, em especial o peso computacional dos métodos; (iii) caracterizar a evolução da herança e da coesão interna das classes (LCOM_HS); e (iv) comparar estatisticamente o período imediatamente anterior à IA generativa (2018–2021) com a Era da IA assistida (2022–atual), contrastando código produzido por desenvolvedores e código gerado com auxílio de agentes de IA. A operacionalização desses objetivos no paradigma *Goal-Question-Metric* é apresentada na Seção 4.

Espera-se obter, como resultados, um retrato empírico das distribuições de acoplamento, complexidade, herança, coesão e tamanho em cada coorte, com testes não paramétricos que indiquem se as diferenças entre épocas são estatisticamente significativas, em especial no contraste entre o período pré-IA e a Era da IA assistida. Também se espera consolidar um pacote de replicação contendo *scripts* de coleta, extração de métricas, notebooks de análise e artefatos de visualização, de modo a tornar o protocolo reproduzível no GitHub.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 discute os trabalhos relacionados; a Seção 4 descreve o tipo de pesquisa, o ambiente, a amostragem, as métricas, os instrumentos, os experimentos realizados e o cronograma do TCC II; a Seção 5 apresenta os resultados parciais já obtidos — filtragem da amostra, distribuição por coortes e piloto de identificação de agentes de IA; e o Apêndice A documenta o planejamento das visualizações longitudinais.

2. Fundamentação Teórica

Esta seção apresenta os conceitos consolidados que embasam o estudo: qualidade interna de *software*, a suíte de métricas de Chidamber e Kemerer, o paradigma *Goal-Question-Metric*, a mineração de repositórios e a inteligência artificial generativa no recorte da Era da IA assistida.

2.1. Qualidade interna e métricas orientadas a objetos

A norma *ISO/IEC 25010* define qualidade de *software* como um conjunto de características que expressam o grau em que um sistema atende a necessidades explícitas e implícitas [ISO/IEC 2011]. Nesse modelo, a qualidade interna refere-se a propriedades do código-fonte observáveis por análise estática, incluindo modularidade, coesão e facilidade de manutenção. Pressman e Maxim destacam que sistemas com melhor qualidade interna tendem a exigir menor esforço de evolução e a permanecer viáveis por mais tempo [Pressman and Maxim 2020].

Para tornar essas características mensuráveis em código orientado a objetos, Chidamber e Kemerer propuseram um conjunto clássico de indicadores em nível de classe [Chidamber and Kemerer 1994]. Acoplamento entre objetos (*Coupling Between Objects* — CBO) descreve o grau de dependência de uma classe em relação a outras. Métodos ponderados por classe (*Weighted Methods per Class* — WMC) agrega a complexidade dos métodos. Profundidade da árvore de herança (*Depth of Inheritance Tree* — DIT) e número de filhos (*Number of Children* — NOC) caracterizam a hierarquia. Resposta de uma classe (*Response for a Class* — RFC) estima o conjunto de métodos potencialmente acionados a partir de uma mensagem. Falta de coesão em métodos (*Lack of Cohesion in Methods* — LCOM) indica o quanto os métodos de uma classe operam sobre conjuntos disjuntos de atributos. Essas métricas passaram a ser amplamente empregadas para caracterizar projeto, manutenção e falhas em sistemas orientados a objetos [Singh et al. 2010, Radjenović et al. 2013, Aniche et al. 2016]. A forma operacional em que serão extraídas neste trabalho, incluindo a ferramenta e as variantes adotadas, é especificada na Seção 4.

Neste estudo, três métricas assumem papel central na investigação do contraste entre código produzido por desenvolvedores e código assistido por IA. **WMC** (*Weighted Methods per Class*) quantifica a carga computacional agregada dos métodos de uma classe e serve como indicador de boas práticas na granularidade dos métodos: métodos de alto custo computacional aumentam o WMC e sinalizam violações ao princípio de responsabilidade única, dificultando testes, reutilização e a manutenção do código — dimensão sobre a qual os assistentes de IA tendem a gerar variações dependendo do estilo de *prompt* e da ferramenta utilizada. **LCOM_{HS}** (*Lack of Cohesion in Methods*, variante Henderson-Sellers) revela se os métodos de uma classe operam sobre conjuntos disjuntos de atributos: valores altos indicam que a classe agrupa responsabilidades distintas, sinal de baixa coesão e acoplamento implícito. **CBO** (*Coupling Between Objects*), em sua dimensão *horizontal*, captura dependências entre módulos de domínio que não são mediadas pelo mecanismo de injeção de dependências do Spring Boot — portanto, toda variação observada nessa dimensão é atribuível diretamente às escolhas arquiteturais do desenvolvedor ou da IA. Em conjunto, WMC, LCOM_{HS} e CBO horizontal formam o eixo de avaliação mais sensível ao estilo de produção de código e, por isso, constituem o foco analítico principal do contraste E3 (pré-IA) versus E4 (Era da IA assistida).

2.2. Goal-Question-Metric

O paradigma *Goal-Question-Metric* (GQM) organiza avaliações de *software* em três níveis: o objetivo do estudo (*goal*), as perguntas que o detalham (*questions*) e as métricas que as respondem (*metrics*) [Basili et al. 1994]. A abordagem evita coletar indicadores sem vínculo com a intenção da pesquisa: primeiro define-se o que se deseja aprender; em seguida, traduz-se esse objetivo em perguntas mensuráveis; por fim, escolhem-se as medidas adequadas. Neste trabalho, o GQM liga o objetivo de analisar a evolução da qualidade estrutural em projetos Spring Boot às dimensões de acoplamento, complexidade, herança, coesão e tamanho, operacionalizadas na Seção 4.

2.3. Mineração de Repositórios de Software

A Mineração de Repositórios de *Software* (*Mining Software Repositories* — MSR) analisa artefatos reais — código-fonte, histórico de versões e metadados de plataformas —

para identificar padrões de desenvolvimento, evolução e qualidade [Hassan 2008]. Em vez de apoiar-se apenas em relatos ou documentação, a MSR observa evidências extraídas dos próprios repositórios. No GitHub, essa estratégia exige filtros de relevância, exclusão de *forks* e consideração de um intervalo de maturação após a criação do repositório, a fim de reduzir ruído e projetos ainda imaturos [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016]. Esses cuidados conceituais orientam a amostragem descrita na metodologia.

2.4. Inteligência artificial generativa

Neste trabalho, inteligência artificial generativa designa sistemas baseados em modelos de linguagem de grande escala, treinados em corpora de texto e código, capazes de produzir ou completar trechos de programa a partir de contexto e de *prompts*. Ferramentas como GitHub Copilot, ChatGPT, Claude Code e Cursor ilustram esse paradigma e delimitam o que se denomina Era da IA assistida, adotada como quarta época histórica do estudo. Experimentos controlados indicam ganhos de produtividade e variações na qualidade do código sugerido por esses assistentes [Peng et al. 2023, Moradi Dakhel et al. 2023], o que motiva investigar se a estrutura interna de projetos Spring Boot publicados no GitHub apresenta sinais observáveis nesse período.

3. Trabalhos Relacionados

Esta seção discute cinco estudos empíricos da engenharia de *software* que tratam da qualidade de código assistido por IA, da correteza de artefatos gerados por modelos de linguagem ou da interpretação de métricas de acoplamento em Java. Para cada trabalho, apresentam-se o objetivo e o método, os resultados pertinentes a este TCC e a relação com a proposta — complementar, comparar ou reutilizar o mesmo tipo de evidência. Os três primeiros artigos foram publicados em 2023 e estão diretamente ligados ao recorte da Era da IA assistida.

A qualidade do código produzido por assistentes foi avaliada de forma experimental com o conjunto HumanEval, comparando GitHub Copilot, Amazon CodeWhisperer e ChatGPT quanto a validade, correção, segurança, confiabilidade e manutenibilidade [Yetistiren et al. 2023]. O método consistiu em gerar soluções a partir de enunciados e inspecionar o código resultante com métricas de qualidade e critérios de aceitação dos testes do *benchmark*. Os resultados indicam taxas de correção de 65,2% para o ChatGPT, 46,3% para o Copilot e 31,1% para o CodeWhisperer, além de dívida técnica média associada a *code smells* da ordem de minutos por trecho. Esse estudo é complementar ao presente trabalho: enquanto avalia trechos gerados em tarefas isoladas, esta pesquisa observa a estrutura de repositórios Spring Boot reais ao longo do tempo, permitindo contrastar o período pré-IA com a Era da IA assistida.

A capacidade do GitHub Copilot como programador-par foi examinada em dois conjuntos de tarefas: problemas algorítmicos fundamentais e um corpus de soluções humanas, corretas e defeituosas [Moradi Dakhel et al. 2023]. O método combinou geração repetida de soluções, verificação de funcionalidade e comparação com implementações de programadores. Os resultados mostram que o Copilot produz soluções para a maioria dos problemas, porém parte delas é defeituosa ou não reproduzível; a taxa de correção humana superou a do assistente, embora o código gerado com falhas tenda a exigir menor esforço de reparo. Os autores concluem que a ferramenta pode ser um ativo para

desenvolvedores experientes e um passivo para novatos. O presente trabalho não replica o desenho de tarefas algorítmicas; em vez disso, complementa essa evidência ao perguntar se a adoção massiva de assistentes, a partir de 2022, coincide com mudanças nas métricas estruturais de projetos Java/Spring Boot publicados no GitHub.

A correção funcional de código gerado por modelos de linguagem, incluindo o ChatGPT, foi avaliada com o *framework* EvalPlus, que amplia o HumanEval com uma quantidade substancialmente maior de casos de teste obtidos por geração automática e mutação [Liu et al. 2023]. O método consistiu em reexecutar 26 modelos populares sobre HumanEval e HumanEval+, medindo *pass@k*. Os resultados mostram quedas de até 19,3%–28,9% em *pass@k* quando os testes adicionais são considerados, e alteração no ranqueamento de modelos — inclusive casos em que o ChatGPT deixa de superar alternativas de código aberto. A relação com este trabalho é de complementaridade: os resultados de [Liu et al. 2023] demonstram que a qualidade funcional do código gerado por IA é frequentemente superestimada em *benchmarks* curtos; o presente estudo desloca o foco da correção de funções sintéticas para a qualidade estrutural de sistemas Spring Boot completos, em coortes históricas.

O impacto da injeção de dependências sobre a manutenibilidade em projetos Java foi investigado com duas métricas ponderadas — DCE e DCBO — implementadas no CKJM-Analyzer, extensão da ferramenta CKJM [Sun et al. 2022]. O método aplicou análise estática a um conjunto de projetos *open source*, distinguindo acoplamento “suave” (via injeção) de acoplamento “rígido” (instanciação direta). Os resultados indicam que a injeção de dependências reduz o acoplamento efetivo quando este é ponderado, o que métricas clássicas de CBO tendem a não capturar integralmente. O presente trabalho compartilha o objeto de estudo — código Java com forte presença de injeção, típica do Spring Boot —, porém não propõe métricas novas: utiliza a suíte CK de forma consistente entre épocas para observar evolução estrutural, reconhecendo a limitação apontada por [Sun et al. 2022] na interpretação do acoplamento.

O efeito do GitHub Copilot sobre a produtividade de desenvolvedores profissionais foi medido em experimento controlado, com tarefa de implementação cronometrada e grupo de controle sem o assistente [Peng et al. 2023]. O método alocou participantes de forma aleatória e comparou tempo de conclusão e taxa de término. Os resultados apontam redução substancial do tempo necessário para completar a tarefa (cerca de 55% mais rápido no grupo com Copilot) e maior taxa de conclusão. Essa evidência fundamenta a delimitação da quarta época histórica (a partir de 2022) como Era da IA assistida. A diferença em relação a este trabalho é o construto medido: o experimento de [Peng et al. 2023] avalia desempenho individual em uma tarefa; aqui se avaliam métricas estruturais de repositórios públicos, sem reivindicar causalidade direta entre o uso de um assistente específico e a qualidade de cada projeto.

A identificação automatizada de contas que realizam commits de forma não humana em repositórios públicos foi sistematizada por [Dey et al. 2020], que propõem a abordagem BIMAN (*Bot Identification by commit Message, commit Association, and author Name*). O método combina três heurísticas independentes — análise de padrões no nome e no endereço de correio eletrônico do autor (BIN), medição de repetitividade das mensagens de commit por alinhamento de sequência (BIM) e análise de características estruturais dos commits por floresta aleatória (BICA) — integradas em um classificador de

ensemble que obteve AUC-ROC mediano de 0,90. Concebido em 2020, o BIMAN identifica com precisão bots de automação clássicos, como *Dependabot*, bots de integração contínua e sincronizadores de código. Sua principal limitação para o presente trabalho é que ferramentas de IA generativa, como GitHub Copilot e Cursor, frequentemente realizam commits sob as credenciais Git do próprio desenvolvedor, sem criar uma identidade de bot própria, o que torna a análise de nome e de mensagem insuficiente para detectar esse padrão de adoção.

A lacuna deixada pelo BIMAN foi enfrentada por [Robbes et al. 2026], que investigam a adoção de agentes de codificação de IA generativa no GitHub em larga escala. O estudo propõe uma metodologia em camadas: no primeiro nível, verifica-se a presença de arquivos de configuração específicos de agentes no repositório — como `CLAUDE.md`, `.cursorrules`, `AGENTS.md` e `.github/copilot-instructions.md` —, cobrindo 93 heurísticas para 63 agentes distintos; quando um repositório não contém esses arquivos mas menciona os diretórios correspondentes no `.gitignore`, configura-se adoção silenciosa. No segundo nível, analisam-se commits recentes em busca de trailers `Co-authored-by` com assinaturas de agentes e de autores com identidade de bot. Com essa abordagem, os autores identificaram adoção em aproximadamente 12% dos repositórios públicos ativos do GitHub, com precisão superior a 99% e custo de uma única requisição de API por repositório no nível de arquivos. Essa metodologia é adotada integralmente no presente trabalho para caracterizar a presença de agentes de IA na amostra de repositórios Java Spring Boot.

Em conjunto, os estudos revisados mostram que assistentes de IA alteram produtividade e qualidade de trechos gerados, que métricas clássicas de acoplamento exigem cautela em sistemas com injeção de dependências, e que já existem metodologias validadas para identificar empiricamente repositórios que adotam agentes de IA no desenvolvimento. Nenhum dos estudos anteriores, contudo, descreve a trajetória longitudinal da qualidade estrutural de projetos Spring Boot no GitHub, nem aplica critérios de identificação de adoção de IA a uma amostra de repositórios Java para enriquecer o contraste entre épocas. Essa lacuna é o ponto de partida da metodologia apresentada a seguir.

4. Materiais e Métodos

Esta seção descreve o tipo de pesquisa, o ambiente e os instrumentos, a amostragem, as métricas de avaliação, os procedimentos estatísticos, os experimentos já realizados e o cronograma do TCC II. O desenho é classificado como **empírico**, porque se baseia em repositórios reais do GitHub; **quantitativo**, porque emprega métricas estruturais e testes estatísticos formais; e **longitudinal**, porque organiza a amostra em quatro coortes temporais definidas pela data de criação do repositório, permitindo contrastar gerações distintas de projetos Spring Boot e isolar o salto entre o período pré-IA (E3) e a Era da IA assistida (E4). A classificação justifica-se pelo objetivo de obter evidências observáveis, comparáveis entre épocas, e não pelo acompanhamento do mesmo repositório ao longo de toda a sua vida.

Importa reconhecer uma limitação do desenho por coortes: repositórios de épocas mais antigas tiveram mais tempo de evolução orgânica do que os criados recentemente. Por isso, diferenças entre épocas não serão interpretadas automaticamente como melhoria ou piora de qualidade, mas como variações associadas à coorte, com controle explícito do



Figura 1. Visao geral da metodologia adotada no estudo

tamanho do projeto (LOC) e da versão do Spring Boot.

A Figura 1 resume o fluxo em três etapas: coleta dos repositórios; extração das métricas com a ferramenta CK; e consolidação para análise estatística e visualização.

4.1. Ambiente e instrumentos

A coleta será realizada em Python, com a *GitHub GraphQL API*, paginação e controle de *rate limit*. Essa interface permite recuperar metadados específicos (*createdAt*, estrelas, linguagens, *fork*) em uma única consulta estruturada, o que torna a amostragem mais reprodutível do que uma sequência de chamadas REST. A análise estrutural será executada com a ferramenta CK, versão 0.7.1, sobre JVM em versão fixa, documentada no pacote de replicação [Aniche 2015]. A CK foi escolhida por calcular, de forma automatizada, a suíte de Chidamber e Kemerer em código Java e por ser amplamente utilizada em estudos de MSR. O Power BI será o ambiente de ETL e de *dashboards* exploratórios. A inferência estatística e as figuras finais reproduzíveis serão conduzidas em *notebooks* Python (*scipy*, *statsmodels*) ou R. Diferentes ferramentas de métricas podem implementar o mesmo indicador de modos distintos, em especial o acoplamento [Child et al. 2019]; por isso a versão da CK, da JVM e os parâmetros de execução permanecerão constantes em todas as épocas.

4.2. Amostragem e procedimentos

A amostra é formada por repositórios públicos escritos principalmente em Java e com evidência de uso de Spring Boot indicada pelo tópico *spring-boot* na API de busca do GitHub. Foram considerados apenas projetos não forkeados, não arquivados, não desabilitados, com pelo menos um *commit* de *push* nos três anos anteriores à data de referência da coleta e que não correspondiam a materiais educacionais ou repositórios sem conteúdo. A linguagem principal Java foi exigida como filtro local sobre o campo *language* da API REST, de modo a concentrar a análise em projetos cuja linguagem predominante é Java. Repositórios sem atividade de *push* desde 17 de setembro de 2023 — corte fixado no momento da primeira consulta de contagem — foram excluídos pelo filtro de atividade recente.

A coleta foi conduzida em Python com a API REST do GitHub, com paginação por janelas anuais de *createdAt* de 2008 a 2026 e controle de *rate limit*. A janela temporal deslizante foi adotada para contornar o limite de 1.000 resultados por consulta imposto pela API de busca do GitHub. A consulta-base utilizada foi `topic:spring-boot fork:true`, que inclui forks deliberadamente para que a exclusão seja quantificada após a coleta. Os filtros aplicados sequencialmente sobre os

registros coletados e seus quantitativos resultantes são apresentados na Seção 5.1 (Tabela 4). Os metadados coletados incluem identificador, nome completo, URL, descrição, tópicos, estrelas, número de forks, datas de criação, atualização e último *push*, tamanho estimado em KB, *branch* padrão, licença e indicadores booleanos de *fork*, template, arquivado e desabilitado. Cada repositório será associado a exatamente uma época com base em `createdAt` (UTC), conforme os critérios da Tabela 5 apresentados na Seção 5.2.

4.3. Identificação de adoção de agentes de IA

Uma vez concluída a amostragem, aplica-se uma etapa de identificação de repositórios que apresentam evidência de uso de agentes de IA no desenvolvimento. O objetivo é caracterizar a presença de adoção de IA generativa na amostra, em especial na coorte E4 (2022–atual), enriquecendo o contraste entre épocas com um dado binário de confirmação: se o repositório tem ou não evidência verificável de agente de IA no ambiente de desenvolvimento.

A metodologia adotada é a proposta por [Robbes et al. 2026], adaptada ao contexto deste estudo. A identificação opera em dois níveis, acionados em sequência do mais eficiente ao mais custoso:

Nível 1 — Arquivos de configuração (*file-level traces*): para cada repositório, recupera-se a árvore de arquivos da *branch* padrão com uma única chamada à API REST do GitHub. Verifica-se a presença de arquivos e diretórios que só existem quando o desenvolvedor configura deliberadamente um agente de IA: `CLAUDE.md` e `.claude/` (Claude Code), `.cursorrules` e `.cursor/rules` (Cursor), `AGENTS.md` e `.codex/` (Codex/OpenAI), `.github/copilot-instructions.md` (GitHub Copilot), `.windsurfrules` (Windsurf), `.aider.conf.yml` (Aider), entre outros, totalizando 14 agentes e mais de 40 heurísticas de caminhos. Adicionalmente, inspeciona-se o `.gitignore` do repositório: a presença de entradas como `.cursor/` ou `CLAUDE.md` nesse arquivo indica adoção silenciosa — o desenvolvedor usa o agente mas opta por não versionar os arquivos de configuração.

Nível 2 — Commits recentes (*commit-level traces*): acionado apenas quando o Nível 1 não produz evidência. Recuperam-se os últimos N commits da *branch* principal e inspecionam-se dois sinais: (i) a presença de trailers `Co-authored-by` nas mensagens de commit com domínios e identificadores conhecidos de agentes (por exemplo, `github-copilot`, `claude@anthropic`, `cursor@anysphere`); e (ii) o campo `login` do autor do commit com padrões de identidade de bot (`github-copilot[bot]`, `coderabbitai[bot]`, `sweep[bot]`).

Um repositório é classificado como positivo para adoção de agente de IA se ao menos uma evidência for encontrada em qualquer dos dois níveis. O resultado registra o nível de detecção, o nome do agente identificado e o arquivo ou *trailer* específico que constituiu a evidência, de modo a tornar a classificação auditável e reproduzível.

4.4. Métricas de avaliação

As métricas extraídas pela CK, em nível de classe e de método, são CBO, RFC, WMC, DIT, NOC, LCOM_HS e LOC. Para LCOM, utiliza-se explicitamente a variante de Henderson-Sellers (LCOM_HS), exportada na coluna `lcom` pela CK, distinta de LCOM1, LCOM2 e LCOM3 [Child et al. 2019, Aniche 2015]. Após a sumarização,

cada repositório será representado por médias: `avg_cbo`, `avg_rfc`, `avg_wmc`, `avg_loc_method`, `avg_dit`, `avg_noc` e `avg_lcom`. CBO e RFC observam acoplamento; WMC e LOC, complexidade e tamanho; DIT e NOC, herança; LCOM_HS, coesão. A escolha justifica-se por duas razões: são indicadores clássicos e comparáveis entre projetos [Chidamber and Kemerer 1994] e mapeiam diretamente os objetivos específicos da introdução. Métricas de complexidade e acoplamento são sensíveis ao tamanho das classes e dos métodos [El Emam et al. 2001]; por isso o LOC entra tanto como dimensão de análise quanto como variável de controle.

A Tabela 1 unifica objetivos específicos, questões de pesquisa e métricas, conforme o GQM (Seção 2.2).

Tabela 1. Objetivos específicos, questões de pesquisa e métricas (GQM).

Objetivo específico	Q	Dimensão	Questão de pesquisa	Métricas
Caracterizar acoplamento entre épocas	Q1	Acoplamento	Como a dependência entre classes evoluiu ao longo das quatro épocas?	CBO, RFC
Caracterizar complexidade e tamanho entre épocas	Q2	Complexidade	A densidade lógica dos métodos e o tamanho do código se alteraram com o tempo e na Era da IA?	WMC, LOC
Caracterizar herança e coesão entre épocas	Q3	Herança e coesão	As hierarquias permaneceram rasas e a coesão interna das classes se manteve estável?	DIT, NOC, LCOM_HS
Contrastar E3 e E4 (Era da IA)	Q4	Contraste IA	O período pós-2022 difere de 2018–2021 nas métricas estruturais?	Todas

Fonte: Elaborada pelos autores.

4.5. Métodos estatísticos

Além do filtro mínimo de 2.000 LOC, serão reportadas as distribuições de LOC e de versão do Spring Boot por época. Antes das comparações principais, aplicar-se-á o teste de Kruskal–Wallis sobre LOC entre épocas. Se diferenças forem detectadas, as análises estruturais serão complementadas por estratificação por faixas de LOC e por análise de sensibilidade com métricas normalizadas (razões por classe ou por KLOC). A versão do Spring Boot será registrada por repositório e tratada como variável confundidora na discussão.

Para cada métrica agregada, aplicar-se-á o teste de Kruskal–Wallis às quatro épocas. Havendo significância global, seguir-se-á o teste de Dunn com correção de Bonferroni para comparações par a par. Para a hipótese da Era da IA, empregar-se-á o teste de Mann–Whitney entre E3 e E4. A correção para comparações múltiplas adotará $\alpha_{\text{adj}} = \alpha/m$, com $\alpha = 0,05$ e m igual ao número de testes da família: $m = 7$ no contraste E3 versus E4; $m = 42$ no conjunto completo de pares de épocas após Kruskal–Wallis (seis pares vezes sete métricas). Serão reportados *p-valores* ajustados, tamanho de efeito (*rank-biserial* ou δ de Cliff) e intervalos descritivos (mediana e amplitude interquartil). As visualizações previstas constam do Apêndice A.

4.6. Experimentos já realizados

Três experimentos foram executados para validar instrumentos e procedimentos antes da análise principal. Os resultados quantitativos obtidos em cada experimento são apresentados na Seção 5.

Piloto de extração CK (novembro de 2025): *scripts* em Python clonaram repositórios Java públicos, invocaram a CK 0.7.1 e consolidaram CBO, RFC, WMC, DIT, NOC, LOC e indicadores auxiliares em CSV. O artefato `analise_repositorios_java.csv` reúne a análise de 520 repositórios, incluindo projetos Spring Boot de ampla adoção, como `spring-projects/spring-boot` (CBO médio 5,45; WMC 4,96; DIT 1,29) e `xkcoding/spring-boot-demo` (CBO 5,05; WMC 2,77; DIT 1,13). O piloto confirma que o ambiente (Python, Git, JVM e CK) está operacional e que as métricas previstas são extraíveis em escala.

Coleta completa da amostra (setembro de 2026): a execução do *pipeline* de coleta e filtragem foi concluída em 20 de setembro de 2026. Os quantitativos de repositórios coletados, os filtros aplicados e o conjunto final selecionado são detalhados na Seção 5.1.

Piloto de identificação de agentes de IA (setembro de 2026): o *script* `detectar_agentes_ia.py` foi executado sobre os 50 repositórios mais populares da amostra (por estrelas), aplicando a metodologia de dois níveis descrita na Subseção 4.3. Os resultados quantitativos e a tabela de agentes identificados são apresentados na Seção 5.3.

4.7. Cronograma do TCC II

O trabalho é desenvolvido em dupla. As Tabelas 2 e 3 apresentam cronogramas quinzenais independentes e complementares, da primeira quinzena de agosto de 2026 à segunda quinzena de novembro de 2026. Gabriel Ferreira concentra-se na amostragem, na mineração GraphQL e no ETL; Guilherme Augusto concentra-se na extração CK, na inferência estatística e nas figuras finais.

Tabela 2. Cronograma quinzenal — Gabriel Ferreira.

Tarefa	A1	A2	S1	S2	O1	O2	N1	N2
Critérios de amostragem e consulta GraphQL	X	X						
Mineração E1–E2 e aplicação dos filtros			X					
Mineração E3–E4 e versão do Spring Boot				X				
ETL no Power BI e limpeza dos dados					X	X		
Dashboards exploratórios e exportação CSV						X	X	
Documentação da coleta e pacote de replicação							X	X

Fonte: Elaborada pelos autores. A1–A2: agosto; S1–S2: setembro; O1–O2: outubro; N1–N2: novembro de 2026.

Tabela 3. Cronograma quinzenal — Guilherme Augusto.

Tarefa	A1	A2	S1	S2	O1	O2	N1	N2
Ambiente CK/JVM e validação do piloto	X							
Pipeline de clonagem e extração CK		X	X					
Extração CK e agregação <i>avg.*</i> (E1–E4)			X	X				
Testes Kruskal–Wallis, Dunn e Mann–Whitney					X	X		
Figuras finais e interpretação estatística						X	X	
Pacote de replicação da análise e revisão do texto							X	X

Fonte: Elaborada pelos autores. A1–A2: agosto; S1–S2: setembro; O1–O2: outubro; N1–N2: novembro de 2026.

5. Resultados Parciais

Esta seção apresenta os resultados das etapas de coleta e experimentação já concluídas: a filtragem da amostra de repositórios Java Spring Boot, a distribuição temporal por épocas históricas e o piloto de identificação de adoção de agentes de IA. Os resultados aqui reportados correspondem às fases preparatórias que antecedem a extração completa das métricas CK e a análise estatística principal, previstas para o TCC II.

5.1. Coleta e filtragem da amostra

A coleta foi conduzida em Python com a API REST do GitHub, com paginação por janelas anuais de `createdAt` de 2008 a 2026 e controle de *rate limit*. Em 17 de setembro de 2026 às 16:36 UTC, a API reportou 95.666 candidatos; ao término da coleta, em 20 de setembro de 2026 às 17:42 UTC, o total era de 95.972. Foram efetivamente coletados 95.894 registros únicos; a diferença de 78 em relação à consulta final reflete variação temporal do índice e da paginação da API, não uma exclusão por critério.

A Tabela 4 detalha o atrito sequencial dos filtros aplicados sobre os 95.894 registros coletados. Cada repositório é contabilizado uma única vez no filtro que o rejeitou primeiro.

Tabela 4. Atrito dos filtros aplicados à amostra de repositórios Java Spring Boot.

Filtro	Antes	Excluídos	Restantes
Linguagem principal diferente de Java ou ausente	95.894	23.298	72.596
Forks (<code>is_fork = true</code>)	72.596	1.089	71.507
Espelhos (<code>is_mirror = true</code>)	71.507	0	71.507
Arquivados (<code>archived = true</code>)	71.507	1.607	69.900
Desabilitados (<code>disabled = true</code>)	69.900	0	69.900
Sem <i>push</i> desde o corte ou sem data válida	69.900	30.856	39.044
Templates (<code>is_template = true</code>)	39.044	238	38.806
Vazios ou sem <i>branch</i> padrão	38.806	3	38.803
Materiais educacionais ou listas identificados	38.803	347	38.456
Total elegível e selecionado	—	57.438	38.456

Fonte: Elaborada pelos autores. Artefato: `runs/20260920_java/filter_attrition.csv`. A soma $57.438 + 38.456 = 95.894$ foi verificada por auditoria automatizada com 13 verificações de integridade, todas aprovadas.

Os 38.456 repositórios elegíveis foram selecionados em sua totalidade, pois não atingiram o teto operacional de 50.000 estabelecido na metodologia. Os metadados coletados incluem identificador, nome completo, URL, descrição, tópicos, estrelas, número de forks, datas de criação, atualização e último *push*, tamanho estimado em KB, *branch* padrão, licença e indicadores booleanos de *fork*, template, arquivado e desabilitado.

5.2. Distribuição da amostra por épocas históricas

Cada repositório foi associado a exatamente uma época com base no campo `createdAt` (UTC). A Tabela 5 apresenta os limites operacionais das quatro coortes e os marcos históricos que as caracterizam.

Tabela 5. Épocas históricas: delimitação operacional das coortes no GitHub.

Época	Início	Fim	Denominação	Marcos representativos
E1	01/01/2009	31/12/2013	Colaboração emergente	Consolidação inicial do GitHub e do ecossistema <i>open source</i> Java.
E2	01/01/2014	31/12/2017	Estruturação e <i>frameworks</i>	Popularização do Spring Boot; difusão de Docker e CI.
E3	01/01/2018	31/12/2021	Maturidade colaborativa	Consolidação de práticas de qualidade e revisão por pares.
E4	01/01/2022	data da coleta	Era da IA assistida	GitHub Copilot, Claude Code, Cursor, ChatGPT e adoção de IA generativa.

Fonte: Elaborada pelos autores. Critério de alocação: campo `createdAt` do repositório no GitHub (limites inclusivos).

A concentração de repositórios na coorte E4 (criados a partir de 2022) era esperada dada a aceleração do uso do GitHub no período recente e o recorte temporal da coleta. Esse desequilíbrio será controlado na análise estatística por estratificação e testes não paramétricos, conforme descrito na metodologia.

5.3. Piloto de identificação de agentes de IA

O *script* `detectar_agentes_ia.py` foi executado sobre os 50 repositórios mais populares da amostra (por número de estrelas), como experimento piloto para validar a metodologia de dois níveis descrita na Seção 4.3. A Tabela 6 consolida os agentes identificados e o nível de detecção correspondente.

Tabela 6. Agentes de IA identificados no piloto com os 50 repositórios mais populares.

Agente identificado	Repositórios	Nível de detecção
Claude Code	8	Nível 1 (arquivos de configuração)
GitHub Copilot	5	Nível 1 (arquivos de configuração)
Codex/OpenAI (AGENTS.md)	5	Nível 1 (arquivos de configuração)
Adoção silenciosa (referência no .gitignore)	2	Nível 1 (extensão via .gitignore)
Outros agentes (<i>trailer</i> de commit)	2	Nível 2 (commits recentes)
Total com evidência de IA	16	—

Fonte: Elaborada pelos autores. Artefato: `resultado_deteccao_ia.csv`. Nota: dois repositórios (`springdoc/springdoc-openapi` e `metasfresh/metasfresh`) foram classificados como adotantes silenciosos por referenciarem diretórios de configuração no `.gitignore` sem incluí-los no repositório.

Dos 50 repositórios analisados, 16 (32,0%) apresentaram evidência verificável de adoção de agentes de IA: 14 detectados no Nível 1 (arquivos de configuração) e 2 no Nível 2 (commits recentes). A taxa de 32% no subconjunto mais popular é consistente com o perfil de projetos maduros com grande base de colaboradores, para os quais a adoção de ferramentas de produtividade é mais frequente. Esse resultado valida a metodologia de dois níveis e confirma que a amostra contém projetos com evidência verificável de uso de IA — condição necessária para que o contraste entre código produzido por desenvolvedores (E3) e código assistido por IA (E4) seja empiricamente testável.

O resultado é particularmente relevante para as questões de pesquisa Q3 e Q4 (Tabela 1): dado que o Spring Boot gerencia o acoplamento vertical por meio de injeção de dependências, as métricas mais diretamente influenciadas pelo estilo de programação do desenvolvedor ou da IA são **WMC** (peso computacional dos métodos), **LCOM_HS** (coesão interna das classes) e **CBO horizontal** (acoplamento entre pacotes de domínio não intermediados pelo *framework*). O piloto estabelece a linha de base para o experimento principal, que medirá essas métricas em escala sobre os 38.456 repositórios selecionados.

Referências

- Aniche, M. (2015). Java Code Metrics Calculator (CK). <https://github.com/mauricioaniche/ck>. Versão 0.7.1.
- Aniche, M., Treude, C., Zaidman, A., van Deursen, A., and Gerosa, M. A. (2016). SATT: Tailoring code metric thresholds for different software architectures. In *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 41–50. IEEE.
- Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The goal question metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. John Wiley & Sons, New York.
- Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

- Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.
- Dey, T., Mousavi, S., Ponce, E., Godfrey, T., Padhy, B., Nguyen, A., and Mockus, A. (2020). Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020)*, pages 209–219, Seoul, Republic of Korea. ACM.
- El Emam, K., Benlarbi, S., Goel, N., and Rai, S. N. (2001). The confounding effect of class size on the validity of object-oriented metrics. *IEEE Transactions on Software Engineering*, 27(7):630–650.
- Hassan, A. E. (2008). The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE.
- ISO/IEC (2011). Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. International Standard ISO/IEC 25010:2011, International Organization for Standardization, Geneva.
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*, pages 92–101, Hyderabad, India. ACM.
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.
- Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates.
- Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., and Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: asset or liability? *Journal of Systems and Software*, 203:111734.
- Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv:2302.06590.
- Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 9 edition.
- Radjenović, D., Heričko, M., Torkar, R., and Živković, A. (2013). Software fault prediction metrics: a systematic literature review. *Information and Software Technology*, 55(8):1397–1418.
- Robbes, R., Hora, A., Valente, M. T., Borges, H., and Ghafari, M. (2026). Agentic much? Adoption of coding agents on GitHub. *ACM Transactions on Software Engineering and Methodology*. Online first.
- Singh, Y., Kaur, A., and Malhotra, R. (2010). Empirical validation of object-oriented metrics for predicting fault proneness models. *Software Quality Journal*, 18(1):3–35.

Sun, P., Kim, D.-K., Hua, M., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9):141.

Yetistiren, B., Ozsoy, I., Ayerdem, M., and Tuzun, E. (2023). Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778.

Apêndice A — Planejamento das visualizações longitudinais

As Tabelas 7 e 8 apresentam o planejamento das visualizações que serão geradas durante a execução do TCC II, relacionando cada figura prevista à respectiva questão de pesquisa, à métrica principal e ao tipo de visualização adotado. O conjunto foi desenhado para permitir a comparação entre as quatro épocas históricas E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022–atual), conforme delimitação operacional na Seção 4, e para evidenciar (i) tendências evolutivas de longo prazo (E1 → E3) e (ii) o eventual impacto da Era da IA assistida (E3 → E4).

Tabela 7. Planejamento das visualizações por questão de pesquisa (parte 1).

Fig.	Q	Métrica	Visualização	Descrição
F1	Q1	CBO	<i>Boxplot</i>	Acoplamento médio entre classes por época (mediana, dispersão e outliers).
F2	Q1	RFC	<i>Boxplot</i>	Métodos potencialmente acionados por classe em cada época.
F3	Q1	CBO, RFC	Série temporal	Medianas de CBO e RFC ao longo das quatro épocas.
F4	Q2	WMC	<i>Boxplot</i>	Complexidade lógica agregada por classe em cada época.
F5	Q2	avg_loc.method	<i>Boxplot</i>	Tamanho médio dos métodos; hipótese de métodos maiores na Era da IA.

Tabela 8. Planejamento das visualizações por questão de pesquisa (parte 2).

Fig.	Q	Métrica	Visualização	Descrição
F6	Q2	avg_wmc, avg_loc.method	Dispersão	Correlação entre tamanho e complexidade médios, colorida por época.
F7	Q3	DIT	<i>Violin plot</i>	Profundidade média da hierarquia de herança por época.
F8	Q3	NOC	<i>Violin plot</i>	Número médio de subclasses diretas por classe.
F9	Q3	LCOM_HS	<i>Boxplot</i>	Ausência média de coesão por repositório em cada época.
F10	Q4	Todas	Painel comparativo E3 vs. E4	Contraste da Era da IA, com <i>p-valores</i> ajustados (Mann–Whitney).

Elaborado pelos autores. Os artefatos finais (PNG, *notebooks* Python/R e arquivo .pbix do Power BI) serão publicados no pacote de replicação previsto para o TCC II.

Para garantir a comparabilidade entre figuras, todas as visualizações adotarão a mesma paleta de cores por época, escala consistente entre painéis e identificação explícita das épocas E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022–atual). Sempre que cabível, os *p-valores* ajustados (Bonferroni) dos testes de Kruskal–Wallis e Mann–Whitney (E3 vs. E4) serão reportados no próprio painel ou em legenda associada.